

Contention-Aware Forecasting of Energy Efficiency through Sequence-Based Models in Modern Heterogeneous Processors

Mohammed Bakr Sikal*, Jeferson González-Gómez*[†], Heba Khdr*, Jörg Henkel*

* Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany

[†] Instituto Tecnológico de Costa Rica (TEC), Cartago, Costa Rica

{bakr.sikal, jeferson.gonzalez, heba.khdr, henkel}@kit.edu

Abstract—We present *EffiCast*, the first methodology for contention-aware energy efficiency forecasting in clustered heterogeneous processors using sequence-based models. Through extensive experimental analysis of energy efficiency sensitivities across core types, voltage/frequency (V/f) levels, application phases, and resource contention scenarios, *EffiCast* uncovers key factors driving energy efficiency variability in modern heterogeneous processors. Leveraging structured data generation and advanced LSTM- and Transformer-based models, *EffiCast* achieves unprecedented accuracy while outperforming state-of-the-art predictive techniques. Deployed on a real heterogeneous processor with Intel’s oneDNN acceleration, *EffiCast* delivers inference latencies as low as 1.82 ms per sequence, enabling seamless integration into proactive resource management frameworks. With the ability to forecast future system states under dynamic workloads, *EffiCast* sets a new standard for energy efficiency optimization in energy-constrained application domains.

I. INTRODUCTION

The demand for energy-efficient computing has never been more critical in modern systems [1], i.e., smart industries, self-driving vehicles, etc., driven by the need to operate within stringent energy budgets while maintaining performance. Concurrently, workloads are also becoming increasingly diverse and complex, exhibiting varying characteristics in terms of computational intensity memory requirements. This diversity complicates the task of achieving energy efficiency, as different applications demand resource management approaches that are tailored to their characteristics.

Heterogeneous processors, such as Intel’s *Alder Lake* Performance Hybrid Architecture [2], have emerged as a promising architectural solution to this challenge, combining high-Performance cores with energy-Efficient ones. To exploit the heterogeneity of such processors, a system-level resource management technique should adjust the mapping of applications to the most suitable core type, depending on their characteristics. However, as will be shown in the next motivational example, the decision-making process for selecting the core type that offers the highest energy efficiency for each application is *not trivial*, especially when combined with V/f level adjustment.

Figure 1 illustrates the execution time and system-wide energy efficiency of two applications from the PARSEC [3] and SPLASH-2 [4] benchmark suites, executed across different V/f levels and core types. The results are normalized against the slowest execution time and the most energy-efficient execution of the same application. All experiments are conducted on a real heterogeneous processor¹. The energy efficiency is quantified as instructions per Joule (IPJ), i.e., the total number of instructions executed per unit of system-wide energy consumption in Joules, where a higher IPJ indicates a more energy-efficient execution. The application *fft* demonstrates a clear sensitivity to both core type and V/f levels, achieving its *fastest* and *most energy-efficient* execution on a P-core. In contrast, the application *dedup* exhibits a clearly different behavior; several executions on E-cores achieve comparable performance and energy efficiency to P-cores. Notably, in the mid-range V/f levels, the E-core execution of *dedup* outperform the P-cores in energy efficiency.

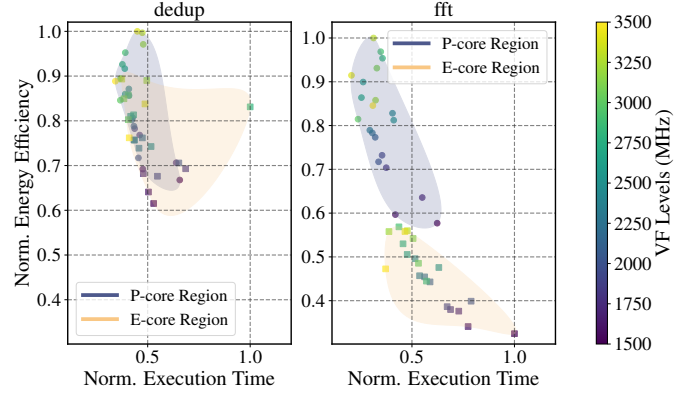


Fig. 1. Executing applications on a heterogeneous processor¹ shows different performance and energy efficiency sensitivities to changes in core types and V/f levels. The same color coding refers to the same VF level across both core types and the different colors represent different VF levels. While *fft* exhibits clear performance and energy efficiency sensitivities to the core types and V/f level changes with a higher energy efficiency on the P-cores, *dedup* shows notably different trends.

This example shows that simple heuristic-based mapping and dynamic voltage and frequency scaling (DVFS) strategies cannot fully leverage the capabilities of modern processors, as it would *fail* to adapt effectively to the nuanced performance and energy characteristics of diverse applications, highlighting the need for more advanced resource management approaches. The state of the art increasingly recognizes this fact. Machine learning (ML)-based and predictive resource management techniques are becoming essential to help proactively predicting the *impact* of core type or/and V/f level selection. For example, prior works have explored code instrumentation [5], [6] as a characterization means of applications. This enabled the imitation learning-based runtime optimization technique in [7] to consider the impact of heterogeneity of core types on energy. However, these methods rely on prior knowledge of applications and require access to their source code, an assumption that is *impractical* in most modern computing environments. Other approaches have employed supervised learning models to predict the performance and power sensitivities of applications [8] to V/f level changes [9], albeit evaluated only in homogeneous processor environments. As will be discussed in Sections I-A and V, predicting energy efficiency sensitivity to *both* the core type and V/f level remains an open challenge in the literature.

A. Challenges of Energy Efficiency Prediction

Predicting energy efficiency in heterogeneous processors is a highly complex problem, driven by the following intertwined challenges. **First**, as shown in Fig. 1, energy and performance sensitivities vary significantly depending on the characteristics of applications and the underlying hardware. Even when known and profiled, it is not trivial to select the optimal core type and V/f level for some applications, e.g., *dedup*. At runtime, however, applications may be unknown, further complicating the task. Without prior knowledge of application

¹A Linux-based heterogeneous Intel® Core™ i9 12th generation Alder Lake processor. Further setup details are explained in Section II.

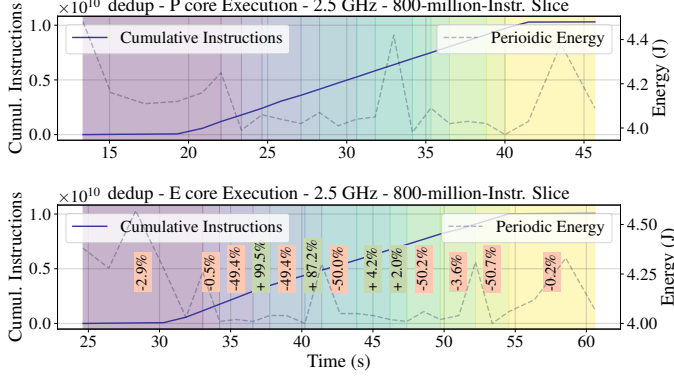


Fig. 2. A zoomed-in view of the phase-level (800 million retired instructions per phase) periodic energy consumption over time for *dedup* shows phase-level energy efficiency improvements (shown in the colored labels) on the E-core compared to the P-core, contrary to the initial observation in Fig. 1, where the overall execution of *dedup* was consistently more efficient on the P-cores.

behavior, accurately predicting energy efficiency becomes a formidable challenge, requiring robust models that are able to generalize across unseen workloads and scenarios that occur at runtime.

Secondly, applications are not monolithic in their behavior; instead, they transition through distinct phases during execution, each exhibiting different resource utilization patterns and sensitivities to system heterogeneity. As we will demonstrate in the analysis section Section II, the same application can display completely different energy efficiency characteristics depending on its execution phase, requiring models to account for this variability.

Third, and even more challenging, is the rapidity at which applications can transition between phases. When predicting the impact of a decision—such as adjusting the V/f level or migrating an application to a different core—the system state may have already evolved by the time the decision is applied. Traditional machine learning prediction techniques that rely on static or *short-sighted* input data fail to capture this temporal dynamic. Instead, as we will demonstrate in Section III, forecasting future system states is crucial to making proactive and effective decisions at runtime.

Finally, heterogeneous processors feature complex *shared* resource hierarchies, including caches and memory bandwidth, which introduce contention both within and across clusters. This contention is highly dynamic, as it depends not only on the co-running applications on the same cluster, but also on the phase transitions of applications running on other clusters. Accurately capturing these contention effects and their impact on energy efficiency adds another layer of complexity to the prediction problem.

To address these challenges, we start by performing an extensive analysis to understand the different factors impacting the predictability of energy efficiency sensitivity of applications to the heterogeneity of modern processors. Guided by the insights from our analysis, we present for the first time *EffiCast*, a novel methodology for contention-aware energy efficiency and sequence-based forecasting for heterogeneous processors. The key contributions of this work are as follows:

- We perform a comprehensive analysis of energy efficiency sensitivities across core types, V/f levels, application phases, and resource contention scenarios, uncovering key factors that impact energy efficiency prediction.
- We propose *EffiCast*, the first methodology for training energy efficiency forecasting models, incorporating structured data generation and sequence-based techniques to predict future system states under dynamic and unseen scenarios.
- We build, train, and evaluate LSTM- and Transformer-based

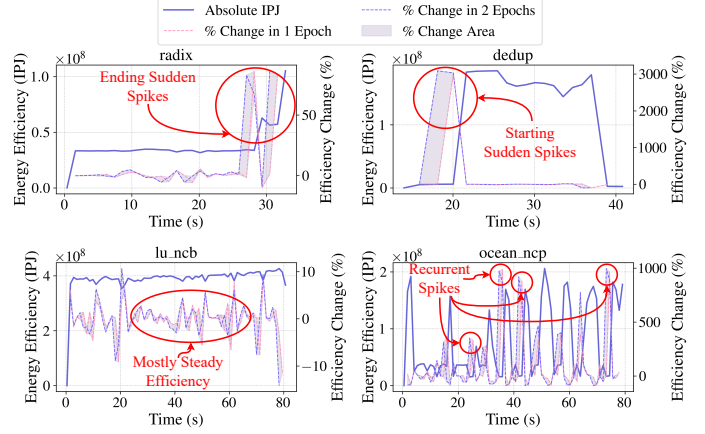


Fig. 3. Absolute energy efficiency (IPJ) and percentage changes over one and two epochs for four representative applications. Shaded areas highlight variability in energy efficiency, reflecting dynamic behaviors such as starting spikes (*dedup*), recurrent spikes (*ocean_ncp*), and abrupt phase transitions (*radix*). These diverse patterns underscore the need for sequence-based models capable of capturing temporal dependencies and dynamic phase transitions.

forecasting models on a real heterogeneous processor platform, demonstrating superior accuracy over traditional state-of-the-art predictive models.

II. ANALYZING ENERGY EFFICIENCY SENSITIVITY TO HETEROGENEITY

To expand on the observations and challenges discussed in Section I, we conduct in the following section a comprehensive analysis of energy efficiency sensitivities across core types, V/f levels, application phases, and resource contention scenarios. Experiments in this section run PARSEC [3] and SPLASH-2 [4] benchmark suites on a Linux-based heterogeneous Intel® Core™ i9 12th generation Alder Lake processor, with Performance Monitoring Counters (PMCs) collected every 100 ms using *perf* [10] and system-wide energy consumption monitored via the Running Average Power Limit (RAPL) interface at the same interval.

A. Volatility of Phase-Level Sensitivities

First, we analyze the phase-level sensitivity of *dedup* to core types and V/f levels, as to compare it with the observations from Fig. 1, when it was executed on a single core type throughout its execution. Fig. 2 illustrates the cumulative retired instructions and periodic energy consumption for *dedup* at 2.5 GHz when executed on both a P-core and an E-core. To enable a phase-level analysis, the retired instructions are divided into slices of 800 million instructions each. As expected, the execution times of different phases vary significantly due to the fluctuating instructions per second (IPS). Since both cores execute the same total number of instructions, slices are matched across both P-core and E-core executions, with color-coding to enable an easy comparison. The highlighted labels show the per-phase *energy efficiency improvements* of the E-core execution compared to the P-core execution for the same phase. While Fig. 1 showed the P-core as the energy-efficient option in general, i.e., considering that the application executes from start to finish on the P-core, Fig. 2 reveals that some phases on the E-core are **up to 2x more energy-efficient** than on the P-core.

At runtime, before changing the application-to-core mapping or the V/f level of a core, a resource management technique should be able to *evaluate* whether such a decision would indeed improve the energy efficiency of the system at that specific phase. To enable this *proactiveness*, there is a need to predict the *phase-level sensitivities* of applications.

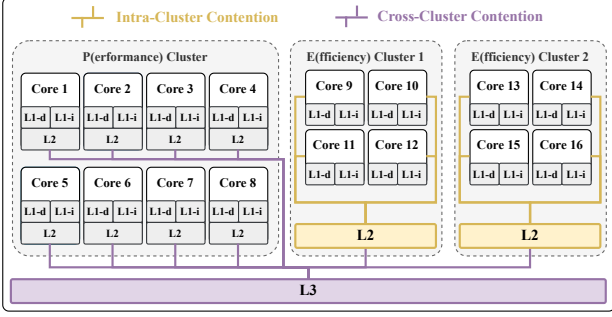


Fig. 4. Illustration of the heterogeneous Intel® Core™ i9 12th generation Alder Lake processor, highlights the complexity of modern memory hierarchies. The shared L2 cache within each E cluster and the shared L3 cache between all clusters create both intra-cluster and cross-cluster contention, respectively.

B. Temporal Variability in Energy Efficiency

The temporal dynamics of energy consumption and performance across different applications lead to intricate patterns and variability in energy efficiency, thereby complicating its prediction at runtime. Figure 3 illustrates the absolute energy efficiency, measured in IPJ, and the percentage changes in energy efficiency over one and two epochs for four representative applications: *radix*, *dedup*, *lu_ncb*, and *ocean_ncp*. The figure was generated by processing periodic data collected during execution, with energy efficiency calculated as the ratio of instructions executed by the application to the total energy consumed by the system.

Figure 3 captures three critical aspects of the temporal variability of energy efficiency over very short time horizons. The dashed lines represent the percentage relative changes in energy efficiency over the next one and two epochs, and provide additional insight into the magnitude and frequency of efficiency variability. The shaded region between the minimum and maximum percentage changes quantifies the potential variability in efficiency. The figure reveals distinct patterns that emerge across the applications. *lu_ncb* exhibits a steady efficiency profile, with very minor fluctuations, maintaining efficiency changes below 10% across its execution. For *radix* and *dedup*, their absolute IPJs remain relatively stable for most of their execution, but exhibit sharp unexpected spikes toward the beginning and end of execution, respectively. The spike in *dedup*'s energy efficiency exceeds a tremendous 3000%. The behavior of *ocean_ncp* is particularly noteworthy, as it displays recurring spikes in energy efficiency, indicative of periodic execution phases with alternating characteristics, with relative IPJ changes of more than 1000%.

C. Intra-Cluster and Cross-Cluster Contention

The complexity of accurately predicting energy efficiency in heterogeneous multi-core processors is further amplified due to dynamic contention at both intra-cluster and cross-cluster levels. Contention arises as co-running applications compete for shared resources, such as caches and memory bandwidth, leading to variable performance and energy behavior.

Figure 4 highlights the shared resource hierarchy of our target processor, illustrating sources of contention. Within clusters, applications compete for resources like L2 caches in E clusters, creating *intra-cluster* contention. Meanwhile, *cross-cluster* contention occurs at shared L3 caches accessed by all cores. This multi-level contention complicates energy sensitivity predictions, as application behavior is influenced by both its own resource usage and that of co-running applications. To explore these effects, we conducted extensive experiments with workloads of up to 16 parallel applications. Figure 5 shows execution times for four applications (*cannal*, *bodytrack*, *blackscholes*, and *water_nsquared*) under varying numbers of background applications.

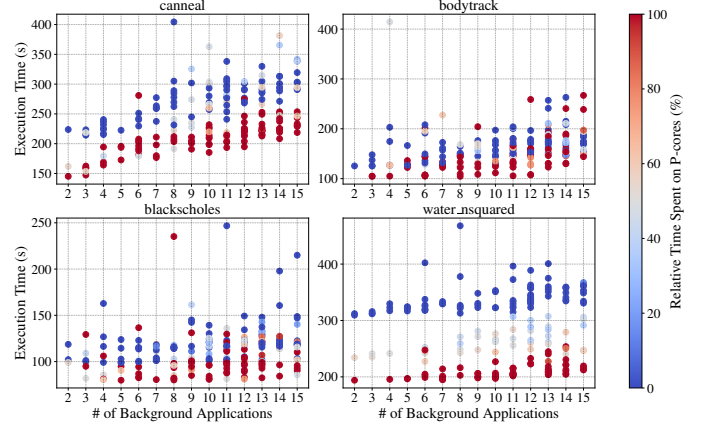


Fig. 5. Execution time of four applications, running with varying numbers of background applications. Random core mapping and periodic migrations reveal the impact of inter- and cross-cluster contention, showing varying but maintained sensitivity across core types with some applications (*cannal*, *water_nsquared*), and significant variability for others (*bodytrack*, *blackscholes*), emphasizing the complexity of contention in heterogeneous processors.

In all experiments, applications were initially mapped randomly to cores, and the application of interest was periodically migrated across different cores types/clusters. Data points are color-coded based on the time spent on P-cores, which typically offer higher performance.

The results reveal several key insights. Execution time does not scale directly with the number of co-running applications, as observed in *cannal* and *water_nsquared*, highlighting that performance degradation depends on the characteristics of co-running applications rather than their number. For *water_nsquared* and *cannal*, P-cores consistently deliver the best performance regardless of contention levels. In contrast, applications like *bodytrack* and *blackscholes* show significant variability between P-core and E-core executions, with performance heavily influenced by both contention levels and migration decisions.

In summary, this analysis demonstrates that accurately modeling energy efficiency sensitivity to modern heterogeneous processors *requires* powerful *forecasting* models capable of capturing *temporal* dynamics of *phase-level sensitivities* under *inter-cluster* and *cross-cluster* contention.

III. EFFICAST: NOVEL FORECASTING METHODOLOGY

Motivated by our findings and observations in Section II, we present for the first time *EffiCast*, a novel methodology for contention-aware energy efficiency sequence-based forecasting for heterogeneous processors. Our problem formulation and training methodology address all the challenges discussed in Section I-A and leverages the observations from the extensive experimental analysis in Section II. In the following subsections, we present our machine learning problem formulation, training data generation approach, feature engineering methodology, and model architectures.

A. Problem Formulation

The forecasting task aims to predict the energy efficiency of a heterogeneous, clustered multi-core processor system under dynamic workload conditions. Formally, let T represent the number of epochs observed, and $\mathcal{X}_t = \{\mathbf{X}_t^i\}$ represent the set of input features at epoch t , where $\mathbf{X}_t^i \in \mathbb{R}^d$ corresponds to individual features for the i -th application and system-level metrics. The system energy efficiency at epoch t is denoted as E_t , expressed in IPJ. Given the input feature sequence $\{\mathcal{X}_t, \mathcal{X}_{t+1}, \dots, \mathcal{X}_{t+T}\}$, the task is to predict the energy efficiency E_{t+T+1} of the system *under a hypothetical configuration change*, $\mathbf{C}_{t+T+1}$, where:

- $\mathbf{C}_{t+T+1}$ includes the new mapping of the application of interest and the adjusted V/f level.

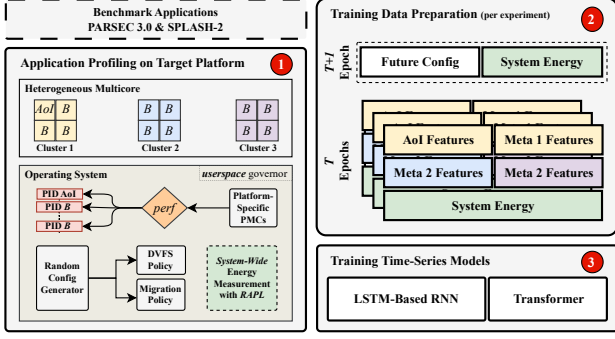


Fig. 6. An overview of EffiCast, our novel methodology for contention-aware energy efficiency sequence-based forecasting for heterogeneous processors.

- $\mathcal{X}_t$ includes features derived from the application of interest, aggregated cluster metrics, and system-wide metrics.

The objective is to minimize the forecasting error $\mathcal{L}$, which quantifies the mean squared difference between the true energy efficiency E_{t+T+1} and the predicted value $\hat{E}_{t+T+1}$ across all N samples.

B. Training Data Generation

To train the forecasting models, we conduct experiments using applications from the PARSEC and SPLASH-2 benchmark suites. Each experiment randomly samples a set of applications and executes them on a Linux-based heterogeneous Intel® Core™ i9 12th generation Alder Lake processor. PMCs are collected every 100 ms using *perf* [10], while system-wide energy consumption is monitored via the RAPL interface. The energy efficiency is expressed in IPJ, representing the total instructions executed per unit of energy consumption. Experiments span V/f levels from 1.5 GHz to 3.5 GHz. Each workload consists of 2 to 16 applications, occupying the 16 cores on the platform. The cores are grouped into three clusters: P cluster, E1 cluster, and E2 cluster. The E clusters share clock domains, while the P cluster supports per-core DVFS adjustments. At every T epochs, one application is designated as the application of interest, migrated to a different core or cluster, and its V/f level adjusted. The energy efficiency at the end of the T epochs is the *label*, while metrics from the preceding epochs and the new configuration, i.e., new mapping and new V/f level, form the *features*.

C. Feature Selection and Engineering

Accurate energy efficiency forecasting in heterogeneous processors requires a structured feature set capturing temporal, spatial, and application-level dynamics. The features represent the behavior of the application of interest (AoI) and its interactions with co-running applications, shared resources, and the broader system environment, derived from three hierarchical levels: individual application metrics, cluster-specific aggregated metrics, and system-wide energy consumption. Individual features include fine-grained PMCs of the AoI, such as instructions executed, cycles, cache accesses, cache misses, branches, and branch misses. Cache-related metrics differ based on core type; for E clusters, they primarily reflect L2 behavior, while for P cores, they capture L3 interactions. This distinction highlights core-specific contention patterns. For instance, E clusters with high L2 miss rates may incur delays from subsequent L3 misses, while P cores directly access main memory upon L3 misses, leading to differing penalties. These metrics capture both execution behavior and resource contention dynamics across the memory hierarchy.

Cluster-specific features model the aggregated behavior of co-running applications. Metrics from applications on the same cluster as the AoI form the "Meta Application 1" group, capturing *intra-cluster contention*, such as L2 cache contention in E clusters or L3

contention in the P cluster. Aggregated metrics from applications on the other two clusters form the "Meta Applications 2 and 3" groups, reflecting *cross-cluster contention*, such as competition for shared L3 caches and memory bandwidth. To model the energy component, system-wide energy consumption is included, reflecting overall energy dynamics under varying workloads. Additionally, the core type and V/f level of the hypothetical next epoch configuration are incorporated to enable predictions under dynamic scenarios, such as task migration or frequency scaling.

Features were selected based on domain knowledge and data-driven analysis, prioritizing metrics with strong correlations to energy efficiency while eliminating redundancy. Preprocessing involved scaling numerical features to zero mean and unit variance and encoding categorical features as integers. For sequence-based modeling, features were arranged into fixed-length sequences representing consecutive epochs, ensuring temporal coherence. The final epoch in the sequence always shows a different core type and V/f level configuration of the AoI. This hierarchical feature representation captures the AoI's behavior, interactions with co-running applications, and impact on system energy efficiency. It also enables the forecasting models to account for temporal dependencies, contention patterns, and dynamic resource utilization, ensuring accurate predictions under diverse and unseen runtime conditions.

D. ML Models

Energy efficiency forecasting in heterogeneous processors requires models capable of capturing temporal dependencies, resource contention patterns, and the impact of dynamic reconfigurations. Traditional approaches, such as decision trees and shallow neural networks, struggle to model these complex relationships. Sequence-based models, specifically Transformers and LSTMs, are well-suited for this task due to their ability to process temporal sequences. Transformers leverage attention mechanisms to capture long-range dependencies, while LSTMs maintain temporal coherence through recurrent connections.

The Transformer model employs an encoder architecture to process sequences of system metrics and predict future energy efficiency. The input sequence $\{\mathbf{X}_t, \mathbf{X}_{t+1}, \dots, \mathbf{X}_{t+T}\}$, augmented with positional encodings, is passed through *Multi-Head Attention* layers to capture temporal dependencies and incorporate parameters of hypothetical configuration changes, such as core type and V/f level. A *Feed-Forward Network* (FFN) refines these features through non-linear transformations:

$$\text{FFN}(\mathbf{u}) = \text{ReLU}(\mathbf{u}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2,$$

where $\mathbf{W}_1, \mathbf{W}_2, \mathbf{b}_1, \mathbf{b}_2$ are learnable parameters. The final encoder output is processed by a regression head to predict the energy efficiency E_{t+T+1} . Key hyperparameters, such as the number of layers, attention heads, and hidden dimensions, are fine-tuned using grid search.

The LSTM model processes sequences by maintaining a hidden state across epochs. The input sequence is passed through an LSTM network, where each cell computes:

$$\mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{X}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h),$$

with $\mathbf{h}_t$ representing the hidden state and $\mathbf{W}_h, \mathbf{U}_h, \mathbf{b}_h$ as learnable parameters. The final hidden state is passed through a regression head to predict E_{t+T+1} . LSTM hyperparameters, including the number of layers, hidden units, and dropout rates, are fine-tuned using random search. The model employs the Adam optimizer with a learning rate scheduler for efficient convergence.

Eventually, both models are trained to minimize mean squared error (MSE), with early stopping to prevent overfitting. As will be demonstrated in the Section IV, these architectures effectively model temporal and spatial dynamics, significantly outperforming traditional baselines in energy efficiency forecasting.

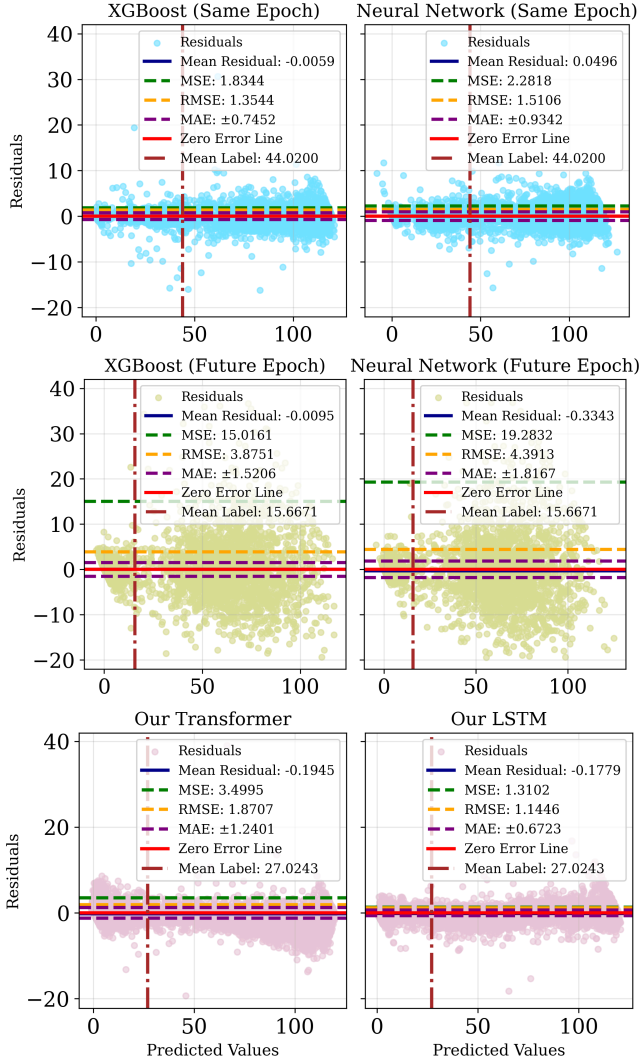


Fig. 7. Our Transformer and LSTM models demonstrate exceptional accuracy in forecasting future energy efficiency under hypothetical system configurations, with RMSE scores as low as 1.14. In contrast, simpler state-of-the-art models, while effective at predicting energy efficiency assuming no change in the immediate next epoch, fail to generalize when tasked with predicting energy efficiency for future epochs. This underscores the critical role of leveraging sequence data and advanced ML models to accurately forecast energy efficiency in dynamic and evolving scenarios.

IV. EVALUATION

A. Experimental Setup and Model Architectures

All experiments and evaluations are conducted on a Linux-based machine with heterogeneous Intel® Core™ i9 12th generation Alder Lake processor. The encoder of the Transformer consists of a multi-head attention mechanism with 8 heads, each using a key dimension of 64, to capture temporal dependencies across epochs. The feed-forward network refines these features through two dense layers, the first with 128 units and ReLU activation, followed by dropout with a rate of 0.1, and the second aligning the output dimensions for residual connections. Dropout is applied after both the attention and feed-forward layers to prevent overfitting, and layer normalization ensures training stability. The final encoder output is passed through a regression head that predicts energy efficiency for the next epoch using the last time step's refined representation. The model is optimized with the *Adam* algorithm at a learning rate of 1×10^{-3} and trained for up

to 100 epochs with early stopping, where validation loss is monitored with a patience of 5 epochs.

The LSTM model employs a stacked architecture with two recurrent layers. The first LSTM layer contains 64 units and outputs a sequence of hidden states, which is further condensed by a second LSTM layer with 32 units, producing only the final hidden state. Dropout regularization with a rate of 0.2 is applied after each LSTM layer to enhance generalization. The final hidden state is passed through a dense regression layer to predict the energy efficiency of the next epoch. The model is also optimized with the Adam algorithm using a learning rate of 1×10^{-3} and trained with the same early stopping criteria as the Transformer.

Both models are trained using a mean squared error (MSE) loss function, with the a dataset of 1.81 million samples, divided into subsets of training (70%), validation (15%) and testing (15%). After experimenting with multiple sequence lengths, we empirically opt for 5-epoch long sequences. The label of the model is the IPJ of the next immediate epoch, computed in millions of instructions per Joule.

B. Model Performance Comparison

As discussed in Section I and Section II-B, state-of-the-art resource management techniques utilizing machine learning focus on predicting the impact of hypothetical configurations on individual system metrics, such as performance, power, or energy. None of these approaches target energy efficiency explicitly, nor do they consider the prediction of future system states on real heterogeneous processors. To evaluate our models, we selected the two most-commonly used machine learning techniques for resource management: Neural Networks (NN) and Boosted Decision Trees (XGB). To ensure a fair comparison against our two models, we train the NN and XGB models under two distinct scenarios.

First, we trained the NN and XGB models to predict the energy efficiency of the *same* epoch using the same features as our models. Second, we trained them to predict the energy efficiency of the *future* (next) epoch, effectively simulating a sequence length of two epochs by using features from the current epoch to predict the next one. Figure 8 presents the residual distributions (actual minus predicted values) for each model, along with the MSE, RMSE, and the means of the target label and residuals.

When tasked with predicting energy efficiency for the *same* epoch, NN and XGB models demonstrated high accuracy, achieving RMSE scores of 1.51 and 1.354, respectively. However, these models struggled to predict energy efficiency for the *future* epoch in scenarios with significant temporal variance, as outlined in Section II-B. This limitation underscores their inability to effectively capture temporal dependencies and adapt to dynamic future system behaviors.

In contrast, our proposed sequence-based models—the Transformer and LSTM—excelled in the *future* task. For forecasting the energy efficiency of the *future* epoch, they achieved exceptionally low RMSE scores of 1.87 and 1.14, respectively, with mean residuals close to zero, demonstrating excellent model fitting.

C. Evaluation of Model Generalization

To evaluate its generalization ability to unseen runtime conditions, we tested our Transformer model on experiments that are *excluded* from the *training set*. The experiment shown in Fig. 8 features 14 applications running in parallel. Among these, we selected the same four applications in Fig. 3: *radix*, *dedup*, *ocean_ncp* and *lu_ncb*, which are known to exhibit diverse temporal sensitivity patterns. Figure 8 illustrates the forecasted and actual energy efficiency values for this scenario, evaluated every five epochs. The results confirm that our models successfully generalize to unseen contention scenarios, maintaining high prediction accuracy even for *ocean_ncb*, which previously showed recurrent energy efficiency spikes throughout its execution. Most importantly, even in the very few instances when our

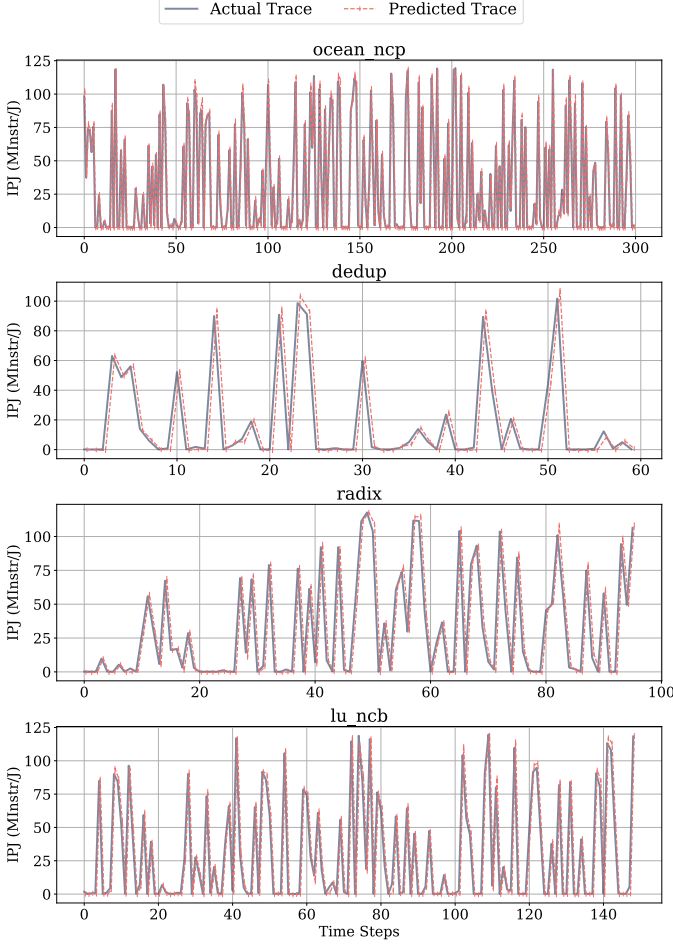


Fig. 8. Actual vs. **forecasted** energy efficiency traces of four applications while running in parallel along with 10 other background applications, in an experiment that is **unseen** during training. Every 5 epochs, the application is migrated and the V/f level of its assigned core is changed. Our Transformer model successfully captures the phase-level sensitivities to these changes along with the contention effects on the chip, delivering a very high prediction accuracy at runtime and enabling robust proactive resource management.

model mispredicted the absolute value of the label, it *never* predicted an *opposite trend* to the actual value in any of the applications across the evaluation experiments, i.e., predicting an increase in energy efficiency when it actually decreases. This demonstrates the robustness of our model architectures in capturing complex temporal and spatial dependencies inherent in heterogeneous multi-core processors. Models with such capabilities will open the door for smarter proactive resource management at runtime that can effectively optimize the energy efficiency of modern computing systems.

D. Training and Inference Overhead

While the proposed Transformer and LSTM models require longer training times compared to classical models such as simpler neural networks and XGB, this overhead is acceptable, as training is performed at design time. Once trained, the models are deployed on the target heterogeneous processor platform and used for runtime inference. Inference is offloaded to Intel’s oneDNN (Deep Neural Network Library), which accelerates neural network computations by leveraging hardware-optimized routines. On our target processor, the average inference time for a single sequence evaluation is 94 ms. However, as the models utilize sequence data, we employ batch inference to process multiple sequences in parallel. This optimization reduces the average inference time drastically to only 1.82 ms per

sequence. This represents less than 2% of the typical 100 ms epoch length, ensuring that the forecasting models impose minimal overhead and are well-suited for real-time energy efficiency optimization in clustered heterogeneous processors.

V. RELATED WORK

A significant body of research has investigated application sensitivity to processor and system characteristics. Early works such as [11] proposed neural network models to optimize application-to-core mappings for minimizing energy consumption, leveraging design-time application profiles. Similarly, [12] used linear regression models to guide runtime application mapping decisions based on these profiles. While these techniques have considered the contention effects on the processor, they have relied on coarse-grained, whole-execution profiles, missing the finer optimization opportunities available through phase-level analysis, as demonstrated previously in Section II.

The importance of phase-level sensitivity was first highlighted in [13], [14], which demonstrated the benefits of adaptive application-to-core mapping adjustments at runtime. Although validated on simulated heterogeneous processors, these studies established the foundation for subsequent approaches. For example, [7] applied imitation learning on a real heterogeneous big.LITTLE platform to optimize performance per watt, using LLVM-based benchmarking to generate training data. Similarly, [5], [15] leveraged LLVM instrumentation to train classifiers that identify phase-level Pareto-optimal configurations at runtime. Compiler-based techniques were also explored in [6], where Java bytecode analysis via the Soot framework triggered runtime configuration changes. In [16], Linux kernel-level tracepoint instrumentation was employed to predict phase-level energy-efficient configurations. However, all these methods depend on source code access and recompilation, making them unsuitable for scenarios where source code is unavailable or recompilation is infeasible.

To address these limitations, performance counters have emerged as a non-intrusive alternative for runtime data collection without requiring source code access. In [9], phase-level performance counter data was used to train machine learning models for predicting application sensitivities to V/f level changes, enabling smart and proactive DVFS at runtime. Similar methods in [17], [18] predicted phase-level performance sensitivities for task migration. However, these works have been validated only in simulated environments, focusing solely on sensitivity to V/f levels or single core types. Furthermore, these methods assume that sensitivity remains static in the immediate subsequent epoch, a limitation demonstrated in Section II.

Recognizing this limitation, [19] introduced LSTM-based forecasting for power consumption. However, their approach was validated only on simulated homogeneous processors, limiting its applicability to real-world heterogeneous systems. In contrast, our proposed EffiCast methodology introduces a novel forecasting approach to predict the energy efficiency sensitivity of applications to both core types and V/f levels on real heterogeneous processors. This approach not only accounts for complex phase-level and contention dynamics but also generalizes to unseen runtime conditions.

VI. CONCLUSION

This paper introduced EffiCast, a novel methodology for forecasting energy efficiency in clustered heterogeneous multi-core processors using Transformers and LSTMs. Unlike simpler state-of-the-art techniques, our models achieved exceptional accuracy capturing complex temporal and resource contention dynamics. Deployed on real hardware with Intel’s oneDNN acceleration, the models demonstrated minimal runtime overhead, with batch inference reducing per-sequence latency to a negligible 1.82 ms. These results validate the integration of energy efficiency forecasting into proactive resource management frameworks, enabling smarter task mapping and DVFS strategies for energy-constrained systems.

REFERENCES

- [1] B. Kocot, P. Czarnul, and J. Proficz, "Energy-aware scheduling for high-performance computing systems: A survey," *Energies*, vol. 16, no. 2, 2023. [Online]. Available: <https://www.mdpi.com/1996-1073/16/2/890>
- [2] "Hybrid architecture (code name alder lake)." [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/hybrid-architecture.htm>
- [3] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The PARSEC Benchmark Suite: Characterization and Architectural Implications," in *Parallel Architectures and Compilation Techniques (PACT)*. ACM, 2008.
- [4] S. C. Woo, M. Ohara, E. Torrie, J. P. Singh, and A. Gupta, "The SPLASH-2 Programs: Characterization and Methodological Considerations," *Int. Symp. Computer Architecture (ISCA)*, 1995.
- [5] U. Gupta, C. A. Patil, G. Bhat, P. Mishra, and U. Y. Ogras, "Dypo: Dynamic pareto-optimal configuration selection for heterogeneous mpsoes," *ACM Trans. Embed. Comput. Syst.*, vol. 16, no. 5s, sep 2017. [Online]. Available: <https://doi.org/10.1145/3126530>
- [6] J. C. R. Da Silva, L. Leão, V. Petrucci, A. Gamatié, and F. M. Q. a. Pereira, "Mapping computations in heterogeneous multicore systems with statistical regression on program inputs," *ACM Trans. Embed. Comput. Syst.*, vol. 20, no. 6, oct 2021. [Online]. Available: <https://doi.org/10.1145/3478288>
- [7] S. K. Mandal, G. Bhat, C. A. Patil, J. R. Doppa, P. P. Pande, and U. Y. Ogras, "Dynamic resource management of heterogeneous mobile platforms via imitation learning," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 12, pp. 2842–2854, 2019.
- [8] J. S. Shim, B. Han, Y. Kim, and J. Kim, "Deepm: Transformer-based power and performance prediction for energy-aware software," in *2022 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2022, pp. 1491–1496.
- [9] M. Rapp, M. B. Sikal, H. Khdr, and J. Henkel, "Smartboost: Lightweight ml-driven boosting for thermally-constrained many-core processors," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 265–270.
- [10] "perf: Linux profiling with performance counters," Jun 2009. [Online]. Available: https://perf.wiki.kernel.org/index.php/Main_Page
- [11] A. Edun, R. Vazquez, A. Gordon-Ross, and G. Stitt, "Dynamic scheduling on heterogeneous multicores," in *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2019, pp. 1685–1690.
- [12] E. Shamsa, A. Kanduri, P. Liljeberg, and A. M. Rahmani, "Concurrent application bias scheduling for energy efficiency of heterogeneous multi-core platforms," *IEEE Transactions on Computers*, vol. 71, no. 4, pp. 743–755, 2022.
- [13] J. Chen and L. K. John, "Efficient program scheduling for heterogeneous multi-core processors," in *Proceedings of the 46th Annual Design Automation Conference*, ser. DAC '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 927–930. [Online]. Available: <https://doi.org/10.1145/1629911.1630149>
- [14] K. Van Craeynest, A. Jaleel, L. Eeckhout, P. Narvaez, and J. Emer, "Scheduling heterogeneous multi-cores through performance impact estimation (pie)," in *2012 39th Annual International Symposium on Computer Architecture (ISCA)*, 2012, pp. 213–224.
- [15] S. K. Mandal, G. Bhat, J. R. Doppa, P. P. Pande, and U. Y. Ogras, "An energy-aware online learning framework for resource management in heterogeneous platforms," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 25, no. 3, may 2020. [Online]. Available: <https://doi.org/10.1145/3386359>
- [16] B. Donyanavard, T. Mück, S. Sarma, and N. Dutt, "Sparta: Runtime task allocation for energy efficient heterogeneous manycores," in *2016 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, 2016, pp. 1–10.
- [17] M. B. Sikal, H. Khdr, M. Rapp, and J. Henkel, "Machine learning-based thermally-safe cache contention mitigation in clustered manycores," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [18] M. Rapp, A. Pathania, T. Mitra, and J. Henkel, "Neural network-based performance prediction for task migration on s-nuca many-cores," *IEEE Transactions on Computers*, vol. 70, no. 10, pp. 1691–1704, 2021.
- [19] M. Sagi, M. Rapp, H. Khdr, Y. Zhang, N. Fafous, N. A. Vu Doan, T. Wild, J. Henkel, and A. Herkersdorf, "Long short-term memory neural network-based power forecasting of multi-core processors," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021, pp. 1685–1690.